题目二：基于多模态信息处理的网络金融舆情风险监测系统

难度等级：中等 **技术方向**：舆情分析、自然语言处理、深度学习

一、项目背景

当前，互联网已经成为金融产品推广和营销的主战场。从微信朋友圈的理财广告，到抖音快手上的投资大师，再到各类论坛中的"内幕消息"，金融信息无处不在。然而，在这些信息中混杂着大量的风险因素：虚假宣传、非法集资、庞氏骗局等等。

与传统的线下集资不同，网络金融舆情具有传播速度快、影响范围广、隐蔽性强的特点。一条包装精美的投资广告可以在几小时内传播到数万人，而监管部门往往难以及时发现和处置。更为棘手的是，这些风险信息以多种形式存在：有的是文字帖子，有的是宣传图片，还有的是短视频。单纯依靠关键词过滤很难全面识别。

从技术角度看，近年来多模态信息处理技术取得了长足进步。我们可以综合运用自然语言处理（NLP）、图像识别、视频分析等技术，对网络舆情进行立体化监测。比如，通过NLP技术识别文字中的高收益承诺、通过OCR技术提取图片中的联系方式、通过视频分析检测营销话术等。

本课题正是希望构建这样一套多模态的网络金融舆情监测系统，能够自动从社交媒体、论坛等平台采集数据，识别其中的金融风险信息，并进行预警。这对于防范网络金融风险、保护群众财产安全具有重要意义。

二、研究目标

本毕业设计的目标是开发一套基于多模态信息处理的网络金融舆情风险监测系统，具体包括：

1. 实现多源舆情数据采集 能够从微博、论坛、贴吧等3-5个主流平台自动采集金融相关的舆情数据，包括文字、图片、视频等多种形式。数据采集需要具备一定的智能性，能够通过关键词、标签等方式筛选出金融相关内容。

2. 构建文本风险识别模型 针对文字类舆情，开发基于NLP的风险识别模型。能够识别出虚假宣传（如"保本保息""日赚千元"）、非法集资特征（如"拉人头""返利"）、诈骗话术（如"内幕消息""限时优惠"）等风险要素。

3. 实现图片内容检测 针对图片类舆情，使用OCR技术提取图片中的文字信息，然后进行风险识别。同时能够识别图片中的金融相关元素，如二维码、微信号、银行卡号等。

4. 建立综合风险评估体系 将文本、图片等多模态信息的分析结果进行融合，对每条舆情进行综合风险评分。评估维度包括：金融属性强度、风险关键词数量、煽动性程度、传播范围等。

5. 开发可视化监测平台 将整个系统封装为Web平台，提供舆情监测、风险预警、数据统计等功能。平台能够实时展示监测结果，对高危舆情进行标注和提醒。

三、技术方案

整个系统可以划分为五个核心模块：数据采集、文本分析、图像分析、综合评估和可视化展示。下面详细介绍每个模块的技术实现方案。

3.1 数据采集模块

这是系统的数据来源，需要从互联网平台获取金融相关的舆情数据。

（1）平台选择 建议选择以下几个平台作为数据源：

- 微博：公开数据较多，API调用相对友好
- 百度贴吧：金融类贴吧活跃度高
- 雪球/东方财富股吧：专业的财经讨论社区
- 小红书：近年来理财内容增多

（2）采集技术 对于提供API的平台，优先使用官方API获取数据，这样更加稳定合规。对于没有API的平台，可以使用网络爬虫技术：

- Scrapy框架：适合大规模的数据采集
- Selenium：适合动态加载的页面

- BeautifulSoup：适合静态页面解析

（3）采集策略 不需要采集全网数据，可以通过以下方式缩小范围：

- 关键词搜索：设定"理财""投资""高收益"等关键词
- 话题关注：监测特定的话题标签
- 用户关注：重点关注一些疑似营销账号

（4）数据存储 采集到的数据需要保存，建议的存储结构：

```
{
  "id": "唯一标识",
  "platform": "平台名称",
  "content": "文本内容",
  "images": ["图片URL1", "图片URL2"],
  "author": "发布者",
  "publish_time": "发布时间",
  "likes": "点赞数",
  "shares": "转发数"
}
```

可以使用MongoDB存储这些半结构化数据，方便后续查询和处理。

3.2 文本分析模块

对采集到的文字内容进行风险识别，这是系统的核心功能之一。

（1）文本预处理 原始文本往往包含噪声，需要先进行清洗：

- 去除HTML标签和特殊符号
- 统一繁简体（转为简体）
- 分词：使用jieba分词库
- 去除停用词（的、了、在等无意义词）

（2）金融属性判断 首先要判断这条舆情是否与金融相关。可以采用两种方法：

方法一：关键词匹配 建立金融关键词库，包括：

- 金融机构：银行、证券、基金、保险
- 金融产品：理财、股票、期货、数字货币
- 金融行为：投资、贷款、融资、众筹

统计文本中包含的金融关键词数量，超过阈值则判定为金融相关。

方法二：文本分类模型 使用机器学习或深度学习模型进行分类。可以使用：

- 传统方法：朴素贝叶斯、SVM
- 深度学习：BERT、RoBERTa等预训练模型

推荐使用BERT，它在中文文本分类任务上效果很好。可以使用开源的中文BERT模型（如哈工大的BERT-wwm），在自己标注的数据集上进行微调。

（3）风险要素识别 对于金融相关的舆情，进一步识别其中的风险要素。可以采用以下几种技术：

① 风险关键词检测 建立风险关键词库，分为几个类别：

- 虚假承诺类：保本、保息、无风险、稳赚不赔
- 高收益诱惑类：日赚千元、月入过万、暴利、翻倍
- 传销特征类：拉人头、下线、分红、返利
- 紧迫营销类：限时、最后名额、内幕消息、错过后悔

统计每个类别的关键词出现次数，作为风险特征。

② 情感分析 识别文本的情感倾向，过度狂热、夸张的表达往往暗示风险。可以使用情感词典或训练情感分类模型。

③ 命名实体识别 提取文本中的机构名、产品名、人名等实体，与已知的非法集资名单进行比对。

（4）煽动性评估 分析文本的煽动性程度，可以从以下几个维度：

- 语气强烈程度（感叹号、极端词的使用）
- 稀缺性暗示（仅剩XX名额）
- 从众压力（已有XX人参与）

3.3 图像分析模块

很多金融舆情会配有图片，比如收益截图、课程海报等。需要对这些图片进行分析。

（1）OCR文字提取 使用OCR技术从图片中提取文字，然后复用文本分析的方法进行风险识别。

推荐工具：

- Tesseract：开源OCR引擎
- PaddleOCR：百度开源的中文OCR工具，效果较好
- 云服务：腾讯云、阿里云的OCR API（付费但准确率高）

提取到文字后，可以检测是否包含联系方式（微信号、QQ号、手机号）、金额数字、收益率等敏感信息。

（2）图像内容识别 除了文字，图片本身的内容也有意义。可以识别：

- 二维码检测：很多营销图片会放二维码引流
- 钱币符号识别：识别¥、$等符号
- 截图识别：判断是否为聊天记录截图、转账截图

这部分可以使用深度学习的目标检测模型，如YOLO或Faster R-CNN。但考虑到实现难度，如果时间有限，可以简化为只做OCR。

（3）图片质量评估 高质量的宣传图片往往更具迷惑性。可以从图片的分辨率、色彩丰富度、是否有设计感等方面进行评估。

3.4 综合评估模块

将文本分析和图像分析的结果进行综合，对每条舆情进行风险评分。

评分维度设计：

1. **金融属性得分**（0-20分）：金融关键词密度
2. **风险关键词得分**（0-30分）：包含风险关键词的数量和类型
3. **煽动性得分**（0-20分）：语气的强烈程度
4. **传播力得分**（0-15分）：点赞、转发等互动数据
5. **实体匹配得分**（0-15分）：是否匹配已知风险实体

总分100分，可以划分为三个风险等级：

- 高危：总分≥70分
- 中危：50-69分
- 低危：<50分

评分公式：

```
风险评分 = 金融属性 × 0.2 + 风险关键词 × 0.3 + 煽动性 × 0.2
          + 传播力 × 0.15 + 实体匹配 × 0.15
```

也可以训练一个机器学习模型来进行评分，使用已标注的样本训练回归模型，直接预测风险分数。

3.5 可视化展示模块

开发一个Web管理平台，提供舆情监测和风险管理功能。

主要页面：

（1）实时监测页面 展示最新采集到的舆情数据，按时间倒序排列。每条舆情显示：

- 内容摘要（前100字）
- 来源平台
- 发布时间
- 风险等级标签（用不同颜色区分）

（2）风险预警页面 专门展示高危舆情，以醒目的方式提醒管理员关注。可以设置预警阈值，当风险分数超过阈值时自动推送通知。

（3）统计分析页面 用图表展示舆情监测的统计数据：

- 每日采集舆情数量趋势（折线图）
- 不同风险等级的占比（饼图）
- 各平台的风险舆情分布（柱状图）
- 高频风险关键词云图（词云）

（4）详情查看页面 点击某条舆情可以查看详细信息，包括：

- 完整内容
- 配图（如有）
- 风险评分明细
- 识别出的风险要素列表

（5）数据管理页面 支持对舆情数据进行管理，如标记误报、添加白名单、导出数据等。

四、实现技术栈

根据上述技术方案，推荐使用以下技术组合：

后端开发

- Python 3.8+：主要开发语言
- Django 或 Flask：Web后端框架
- Celery：异步任务队列，用于定时采集数据
- Redis：消息队列和缓存

数据采集

- Scrapy：爬虫框架
- Selenium：动态页面爬取
- requests：HTTP请求库

自然语言处理

- jieba：中文分词
- HanLP（可选）：更强大的NLP工具包
- transformers：使用BERT等预训练模型
- SnowNLP：情感分析

图像处理

- PaddleOCR：文字识别
- OpenCV：图像预处理
- Pillow：图像操作

数据存储

- MongoDB：存储舆情数据
- MySQL：存储结构化数据（用户、配置等）

前端开发

- Vue.js 3：前端框架
- Element Plus：UI组件库
- ECharts：数据可视化

部署运维

- Linux服务器（Ubuntu/CentOS）
- Nginx：Web服务器
- Docker：容器化部署（可选）
- Supervisor：进程管理

五、核心功能模块详解

5.1 数据采集模块

主要功能：

- 定时自动采集各平台的金融舆情
- 支持关键词、话题等多种采集策略
- 数据去重和增量更新
- 异常处理和日志记录

关键实现思路（以微博为例）：

```python
import scrapy

class WeiboSpider(scrapy.Spider):
    name = 'weibo'

    def __init__(self, keywords, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.keywords = keywords

    def start_requests(self):
        # 构造搜索URL
        for keyword in self.keywords:
            url = f'https://s.weibo.com/weibo?q={keyword}'
            yield scrapy.Request(url, callback=self.parse)

    def parse(self, response):
        # 解析页面，提取微博内容
        for item in response.css('.card-wrap'):
            content = item.css('.txt::text').get()
            author = item.css('.name::text').get()
            time = item.css('.time::text').get()

            yield {
                'platform': 'weibo',
                'content': content,
                'author': author,
                'publish_time': time
            }
```

5.2 文本分析模块

主要功能：

- 文本预处理和分词
- 金融属性分类
- 风险关键词检测
- 情感和煽动性分析

关键实现思路:

```python
import jieba
from transformers import BertTokenizer, BertForSequenceClassification

class TextAnalyzer:
    def __init__(self):
        # 加载BERT模型
        self.tokenizer = BertTokenizer.from_pretrained('bert-base-chinese')
        self.model = BertForSequenceClassification.from_pretrained('bert-base-chinese')

        # 加载风险关键词库
        self.risk_keywords = self.load_risk_keywords()

    def analyze(self, text):
        # 1. 判断金融属性
        is_finance = self.classify_finance(text)

        if not is_finance:
            return {'is_finance': False}

        # 2. 检测风险关键词
        risk_words = self.detect_risk_keywords(text)

        # 3. 评估煽动性
        agitation_score = self.evaluate_agitation(text)

        # 4. 计算风险分数
        risk_score = self.calculate_risk_score(risk_words, agitation_score)

        return {
            'is_finance': True,
            'risk_score': risk_score,
            'risk_words': risk_words,
            'agitation_score': agitation_score
        }

    def detect_risk_keywords(self, text):
        words = jieba.cut(text)
        found_keywords = []

        for word in words:
            if word in self.risk_keywords:
                found_keywords.append(word)

        return found_keywords
```

5.3 图像分析模块

主要功能:

- 从URL下载图片
- OCR文字提取
- 二维码检测
- 敏感信息识别（手机号、微信号等）

关键实现思路:

```python
from paddleocr import PaddleOCR
import re

class ImageAnalyzer:
    def __init__(self):
        self.ocr = PaddleOCR(use_angle_cls=True, lang="ch")

    def analyze(self, image_path):
        # OCR提取文字
        result = self.ocr.ocr(image_path)

        # 合并识别出的文字
        text = ''
        for line in result:
            for word_info in line:
                text += word_info[1][0]

        # 检测敏感信息
        phone_numbers = self.extract_phone(text)
        wechat_ids = self.extract_wechat(text)

        # 检测风险关键词
        risk_words = self.detect_risk_keywords(text)

        return {
            'ocr_text': text,
            'phone_numbers': phone_numbers,
            'wechat_ids': wechat_ids,
            'risk_words': risk_words
        }

    def extract_phone(self, text):
        pattern = r'1[3-9]\d{9}'
        return re.findall(pattern, text)

    def extract_wechat(self, text):
        # 微信号规则：6-20位，字母开头，可包含数字、字母、下划线
        pattern = r'[a-zA-Z][a-zA-Z0-9_-]{5,19}'
        return re.findall(pattern, text)
```

5.4 综合评估模块

主要功能:

- 整合文本和图像的分析结果
- 多维度风险评分
- 风险等级分类
- 生成评估报告

关键实现思路:

```python
class RiskEvaluator:
    def evaluate(self, text_result, image_results):
        # 初始化各项分数
        finance_score = 0
        keyword_score = 0
        agitation_score = 0
```

```python
    contact_score = 0

    # 文本分析结果
    if text_result.get('is_finance'):
        finance_score = 20
        keyword_score = min(len(text_result['risk_words']) * 5, 30)
        agitation_score = text_result['agitation_score']

    # 图像分析结果
    for img_result in image_results:
        # 如果图片包含联系方式，增加风险分
        if img_result['phone_numbers'] or img_result['wechat_ids']:
            contact_score = 15

        # 图片中的风险关键词也计入
        keyword_score += min(len(img_result['risk_words']) * 3, 15)

    # 总分计算
    total_score = finance_score + keyword_score + agitation_score + contact_score
    total_score = min(total_score, 100)  # 最高100分

    # 风险等级
    if total_score >= 70:
        risk_level = 'high'
    elif total_score >= 50:
        risk_level = 'medium'
    else:
        risk_level = 'low'

    return {
        'total_score': total_score,
        'risk_level': risk_level,
        'detail_scores': {
            'finance': finance_score,
            'keyword': keyword_score,
            'agitation': agitation_score,
            'contact': contact_score
        }
    }
```

六、预期成果

完成本毕业设计后，应该能够交付以下成果：

1. 系统软件

- 一套完整的网络金融舆情监测平台
- 包含数据采集、分析、展示三大模块
- 支持3-5个主流平台的舆情监测

2. 数据资源

- 建立金融风险关键词库（500+条）
- 积累舆情样本数据（1000+条）
- 标注的训练数据集（用于模型训练）

3. 模型效果

- 金融属性识别准确率≥80%

- 风险等级分类准确率≥75%
- OCR文字识别准确率≥85%

七、实施难度与可行性分析

7.1 技术难点

难点1：数据采集的稳定性 很多平台都有反爬虫机制，频繁访问可能导致IP被封。部分平台需要登录才能查看完整内容。

解决方案:

- 设置合理的访问频率，避免过于频繁
- 使用代理IP池
- 模拟正常用户行为（设置User-Agent、Cookie等）
- 优先选择对爬虫友好的平台，如微博的公开数据
- 实在实现不出来可以借用本实验室的爬虫平台，已支持对境内常见舆情平台的数据采集

难点2：NLP模型的训练 使用BERT等深度学习模型需要一定的标注数据，数据标注工作量较大。

解决方案:

- 可以先使用开源的预训练模型进行零样本或少样本学习
- 标注100-200条样本进行微调即可获得不错效果
- 如果实在困难，可以降级为使用关键词匹配的方式

难点3：多模态信息融合 如何合理地将文本、图像的分析结果综合起来是一个挑战。

解决方案:

- 采用加权平均的方式进行融合
- 参考已有的风险评估模型，设计合理的权重
- 通过实验不断调整参数

7.2 降低难度的替代方案

如果在实施过程中遇到困难，可以考虑以下简化方案：

方案1：只分析文本，不分析图像 跳过图像分析模块，专注于文本的风险识别，这样可以减少技术难度。

方案2：只采集1-2个平台的数据 减少数据源，降低爬虫开发的工作量。可以选择微博或贴吧这类数据获取相对容易的平台。

方案3：使用规则而非模型 不训练机器学习模型，采用关键词匹配和规则判断的方式进行风险识别。虽然准确率会降低，但实现更简单。

方案4：使用云服务API 对于NLP和OCR等技术，可以直接调用阿里云、腾讯云的API，不需要自己训练模型。但这会产生一定的费用。